nanoGPT / Zero to Hero - Study Notes

Covered so far: imports -> attention -> MLP -> Block -> GPT constructor -> forward pass -> logits -> loss (up to the flattening / cross_entropy line).

1) Big picture

- Goal: predict the next token from previous tokens.
- Main flow: token IDs -> embeddings -> transformer blocks -> logits -> loss.
- Shapes you should always track: (B, T) -> (B, T, C) -> (B, T, vocab_size).

2) Core Python / PyTorch tools

- @dataclass: compact config object for hyperparameters.
- nn.Module: base class for learnable PyTorch modules; gives parameters(), state_dict(), to(), train(), eval().
- nn.Linear: learned affine map on the last dimension only.
- nn.Embedding: learnable lookup table; integer IDs -> embedding vectors.
- nn.ModuleDict / nn.ModuleList: register modules inside dict/list containers so PyTorch tracks them.
- nn.LayerNorm: normalize each token's feature vector (last dim) independently.
- nn.GELU: nonlinearity; makes the MLP more expressive.
- nn.Dropout: randomly zeros activations during training only.
- F.scaled_dot_product_attention: optimized attention kernel in PyTorch.
- F.cross_entropy: logits + target IDs -> one scalar loss.

```
Useful syntax reminders:
- assert condition: stop immediately if condition is false.
- f"..." : formatted string with {expressions}.
- view(-1, ...): infer that dimension automatically.
- size(-1): last dimension.
- tensor.device / tensor.dtype: tensor attributes.
```

3) Causal self-attention

- Input x shape: (B, T, C) = batch, tokens, channels/embedding dim.
- One Linear layer makes QKV together: C -> 3C.
- Split QKV into q, k, v; each has shape (B, T, C).
- Reshape to heads: (B, T, nh, hs) then transpose to (B, nh, T, hs).
- hs = head size = C // nh. The assert checks C is divisible by nh.
- Attention scores are Q @ K^T / sqrt(hs), then causal mask, then softmax.
- Causal mask blocks future tokens by setting illegal scores to -inf before softmax.
- Attention output = attention weights @ V, shape stays (B, nh, T, hs).
- Merge heads back: transpose -> contiguous() -> view(B, T, C).
- Final c_proj mixes head outputs; shape stays (B, T, C).

```
Attention mental model:
Q = what I am looking for
K = what each token advertises
V = the information to copy

Attention = match Q with K, turn matches into weights, use weights to mix V.
```

4) MLP (feed-forward network)

- MLP acts on each token independently; it does not mix information across tokens.
- Structure: Linear(C -> 4C) -> GELU -> Linear(4C -> C).
- Purpose: give each token extra nonlinear processing after attention has gathered context.
- The 4x expansion is temporary extra capacity, then compressed back to C.
- Without GELU, stacked Linear layers collapse into one Linear layer (still linear).

5) Transformer Block

- One block = LayerNorm -> Attention -> residual add -> LayerNorm -> MLP -> residual add.
- Pre-LN design: normalize before each major sublayer for stable training.
- Residual connection means: keep original x + add learned change.
- Shape stays (B, T, C) through the whole block.

```
Block flow:
x = x + attn(ln_1(x))
x = x + mlp(ln_2(x))
```

6) GPT model construction

- wte = token embedding table: (vocab_size, n_embd). Lookup by token IDs.
- wpe = position embedding table: (block_size, n_embd). Lookup by token positions.
- Token embedding answers 'what token is it?'; position embedding answers 'where is it?'.
- x = tok_emb + pos_emb combines identity + position; shape stays (B, T, C).
- drop = Dropout after embeddings; active in training, off in eval().
- h = ModuleList of N Block objects; GPT depth comes from repeating blocks.
- ln_f = final LayerNorm after the last block.
- lm_head = Linear(C -> vocab_size, bias=False); it turns hidden states into vocabulary scores.
- lm_head output shape is (B, T, vocab_size); these raw scores are logits.
- Bias is disabled in lm_head; later the embedding and output weights are tied.

7) Forward pass

- forward(idx, targets=None): same function for training and generation.
- idx is token IDs with shape (B, T). targets is optional; None means generation/inference.
- B, T = idx.size() just reads batch size and sequence length.
- assert T <= block_size prevents sequences longer than the learned position table.
- pos = torch.arange(0, T, dtype=torch.long, device=idx.device) creates position IDs on the same device as idx.
- dtype=torch.long is required because embeddings need integer IDs.
- tok_emb = wte(idx) -> (B, T, C). pos_emb = wpe(pos) -> (T, C).
- x = tok_emb + pos_emb uses broadcasting across batch.
- x flows through every Block in h, then ln_f, then lm_head.
- logits shape = (B, T, vocab_size).
- If targets exist, loss is computed; otherwise loss stays None.
- return logits, loss lets the same forward() support training and text generation.

8) Loss line and flattening

- cross_entropy expects predictions shaped like (N, V) and targets shaped like (N,).
- GPT starts with logits shape (B, T, V) and targets shape (B, T).
- logits.view(-1, logits.size(-1)) flattens B and T into one dimension: (B*T, V).
- targets.view(-1) flattens targets to (B*T,).
- -1 in view() means 'infer this dimension automatically'.
- size(-1) means 'last dimension'. The two -1s have different meanings.
- cross_entropy compares each row of logits with the matching target ID and returns one scalar loss.

```
Loss view pattern:
logits  : (B, T, V) -> (B*T, V)
targets : (B, T)    -> (B*T,)
```

9) Quick shape cheat sheet

Object	Typical shape	Meaning
idx	(B, T)	token IDs
tok_emb	(B, T, C)	token embeddings
pos_emb	(T, C)	position embeddings
x after add	(B, T, C)	token + position info
attention output	(B, nh, T, hs)	per-head output
merged attention	(B, T, C)	heads stitched back together
MLP output	(B, T, C)	token-wise nonlinear transform
logits	(B, T, vocab_size)	raw next-token scores

10) Remember these one-liners

- Attention = tokens communicate with each other.
- MLP = each token thinks alone, with nonlinearity.
- LayerNorm = normalize each token's feature vector.
- Embedding = integer ID -> learned vector lookup.
- Logits = raw scores for every vocabulary token.
- Loss = how wrong the logits are versus the targets.